

1. Problem Statement

A financial application calculates the final amount payable by a customer after applying a discount and a service charge. The application receives the original amount as an integer and the discount and service charge as decimal percentages.

The final amount is calculated as:

$$\text{Final Amount} = \text{Original Amount} - \text{Discount} + \text{Service Charge}$$

where:

$$\text{Discount} = \text{Original Amount} \times \text{Discount Percentage} / 100$$

$$\text{Service Charge} = \text{Amount after Discount} \times \text{Service Charge Percentage} / 100$$

Write a Java program to calculate and display the final amount rounded to two decimal places. The program should use appropriate primitive data types to avoid loss of decimal precision during the calculation.

Input Format

The first line contains an integer representing the original amount.

The second line contains a decimal value representing the discount percentage.

The third line contains a decimal value representing the service charge percentage.

Output Format

Print the final amount rounded to two decimal places.

Code Constraints

$$1 \leq \text{Original Amount} \leq 1000000$$

$$0 \leq \text{Discount Percentage} \leq 100$$

$$0 \leq \text{Service Charge Percentage} \leq 20$$

Sample Input

5000

10.5

5.0

Sample Output

4712.50

2. Problem Statement

A manufacturing system records the number of products produced during N shifts. To identify production performance, the system calculates the total production, average production, and the difference between the highest and lowest production values.

Write a Java program to read the production values into an integer array and calculate these three values. The average production should be displayed as a decimal value rounded to two decimal places.

Input Format

The first line contains an integer N.

The second line contains N space-separated integers representing production quantities.

Output Format

Print the total production on the first line.

Print the average production on the second line.

Print the difference between the maximum and minimum production on the third line.

Code Constraints

$1 \leq N \leq 1000$

$0 \leq \text{Production Quantity} \leq 100000$

Sample Input

5

420 510 380 600 490

Sample Output

2400

480.00

220

3. Problem Statement

A digital security system receives a positive integer identification number. For verification, it needs to calculate the sum of its even digits and the sum of its odd digits separately.

Write a Java program to process the number digit by digit using arithmetic and modulus operations. Display the sum of even digits and the sum of odd digits.

Input Format

The input consists of a single positive integer N.

Output Format

Print the sum of even digits on the first line.

Print the sum of odd digits on the second line.

Code Constraints

$10 \leq N \leq 999999999$

Sample Input

583241

Sample Output

6

21

4. Problem Statement

A warehouse system stores package weights in an integer array. Due to a change in the measurement system, every package weight must be converted from kilograms to grams before generating the report.

Write a Java program to read the package weights and display their corresponding values in grams. The calculation should use an appropriate data type because the converted value may exceed the normal integer range for larger inputs.

Input Format

The first line contains an integer N.

The second line contains N space-separated integers representing package weights in kilograms.

Output Format

Print the converted weights in grams separated by a single space.

Code Constraints

$1 \leq N \leq 1000$

$1 \leq \text{Weight} \leq 1000000$

Sample Input

4

12 25 40 8

Sample Output

12000 25000 40000 8000

5. Problem Statement

A shopping application stores information about a product using a Java class. Each product has a product ID, product price, and available quantity.

Create a class named Product containing these attributes and a method named calculateValue() that returns the total inventory value:

Total Value = Price × Quantity

Create a Product object using the input values and display the calculated inventory value.

Input Format

The first line contains the product ID.

The second line contains the product price.

The third line contains the available quantity.

Output Format

Print the total inventory value rounded to two decimal places.

Code Constraints

$1 \leq \text{Product ID} \leq 100000$

$0 < \text{Price} \leq 1000000$

$0 \leq \text{Quantity} \leq 100000$

Sample Input

105

1250.50

8

Sample Output

10004.00

6. Problem Statement

A data processing system receives N integer values. The system needs to rearrange the array so that all even numbers appear first, followed by all odd numbers, while preserving their original order within each group.

Write a Java program to perform this rearrangement without using another array.

Input Format

The first line contains an integer N.

The second line contains N space-separated integers.

Output Format

Print all even values followed by all odd values, separated by a single space.

Code Constraints

$$1 \leq N \leq 1000$$

$$-10000 \leq \text{Value} \leq 10000$$

Sample Input

8

7 4 9 2 6 11 8 5

Sample Output

4 2 6 8 7 9 11 5

7. Problem Statement

A text-processing application receives a sentence and needs to determine whether the sentence is a palindrome after ignoring spaces and differences between uppercase and lowercase letters.

For example, "Never Odd Or Even" should be considered a palindrome.

Write a Java program to perform the required string processing and print Palindrome if the sentence satisfies the condition; otherwise print Not Palindrome.

Input Format

The input consists of a single line containing a sentence.

Output Format

Print Palindrome or Not Palindrome.

Code Constraints

$$1 \leq \text{Length of sentence} \leq 200$$

The sentence may contain uppercase letters, lowercase letters, and spaces.

Sample Input

Never Odd Or Even

Sample Output

Palindrome

8. Problem Statement

An examination system receives the marks of N students. A method named `calculateGrade(int marks)` must be created to determine the grade of each student according to the following rules:

90–100 → A

75–89 → B

60–74 → C

40–59 → D

Below 40 → F

Write a Java program that uses the method for every student's marks and displays the corresponding grades in the same order.

Input Format

The first line contains an integer N.

The second line contains N space-separated marks.

Output Format

Print the grades separated by a single space.

Code Constraints

$1 \leq N \leq 100$

$0 \leq \text{Marks} \leq 100$

Sample Input

6

92 76 65 48 35 88

Sample Output

A B C D F B

9. Problem Statement

A hospital maintains the token numbers of patients waiting for registration using an `ArrayList`. During registration, some token numbers are cancelled.

Write a Java program to store the token numbers in an ArrayList, remove all token numbers that are divisible by a given cancellation number, and display the remaining token numbers in their original order.

Input Format

The first line contains an integer N.

The second line contains N space-separated token numbers.

The third line contains the cancellation number.

Output Format

Print the remaining token numbers separated by a single space.

If no token remains, print -1.

Code Constraints

$1 \leq N \leq 1000$

$1 \leq \text{Token Number} \leq 100000$

$1 \leq \text{Cancellation Number} \leq 100$

Sample Input

8

12 15 18 21 24 25 30 31

3

Sample Output

15 25 31

10. Problem Statement

A logistics system maintains container IDs using a Vector. A new container is inserted after a specified existing container ID.

Write a Java program to search for the specified container ID in the Vector. If the ID is found, insert the new container ID immediately after it. If the ID is not found, print Container Not Found.

Input Format

The first line contains an integer N.

The second line contains N space-separated container IDs.

The third line contains the existing container ID.

The fourth line contains the new container ID.

Output Format

Print the updated Vector if the existing ID is found.

Otherwise, print Container Not Found.

Code Constraints

$1 \leq N \leq 1000$

$1 \leq \text{Container ID} \leq 1000000$

Sample Input

5

201 305 410 512 620

410

999

Sample Output

201 305 410 999 512 620

11. Problem Statement

A task management application maintains pending task IDs using a LinkedList. The system receives a task ID that has been completed and must remove its first occurrence from the list.

Write a Java program to search for the given task ID and remove its first occurrence. If the task ID does not exist, print Task Not Found.

Input Format

The first line contains an integer N.

The second line contains N space-separated task IDs.

The third line contains the task ID to be removed.

Output Format

Print the updated list if the task exists.

Otherwise, print Task Not Found.

Code Constraints

$1 \leq N \leq 1000$

$1 \leq \text{Task ID} \leq 1000000$

Sample Input

7

101 205 310 205 415 520 610

205

Sample Output

101 310 205 415 520 610

12. Problem Statement

A university receives registration records from different departments. The same student may register more than once due to repeated attempts.

Using a HashSet, determine whether the number of unique student IDs is greater than half of the total number of registration records.

Print High Unique Registration if the condition is satisfied; otherwise print Low Unique Registration.

Input Format

The first line contains an integer N.

The second line contains N student IDs.

Output Format

Print the appropriate message.

Code Constraints

$1 \leq N \leq 1000$

$1 \leq \text{Student ID} \leq 1000000$

Sample Input

8

101 102 103 101 104 105 106 102

Sample Output

High Unique Registration

13. Problem Statement

A company maintains employee performance scores. The scores must be stored in a TreeSet so that duplicate scores are automatically removed and the remaining scores are maintained in sorted order.

The management wants to find the highest score that is strictly less than a given target score.

Write a Java program using TreeSet to perform this operation. If no such score exists, print -1.

Input Format

The first line contains an integer N.

The second line contains N space-separated scores.

The third line contains the target score.

Output Format

Print the greatest score smaller than the target score.

Code Constraints

$1 \leq N \leq 1000$

$0 \leq \text{Score} \leq 100$

$0 \leq \text{Target Score} \leq 100$

Sample Input

8

45 72 88 65 72 91 54 80

75

Sample Output

72

14. Problem Statement

An inventory system maintains product IDs and their quantities using a HashMap. A manager wants to calculate the total quantity available for a particular product category represented by a set of product IDs.

Write a Java program to store product ID and quantity pairs in a HashMap. Given M product IDs, calculate the total quantity of those products. If a product ID is not present in the map, its quantity should not be included.

Input Format

The first line contains N.

The next N lines contain product ID and quantity.

The next line contains M.

The following line contains M product IDs.

Output Format

Print the total quantity of the specified products.

Code Constraints

$1 \leq N \leq 1000$

$1 \leq M \leq 100$

$1 \leq \text{Product ID} \leq 100000$

$0 \leq \text{Quantity} \leq 10000$

Sample Input

5

101 20

102 35

103 15

104 50

105 25

4

101 103 105 110

Sample Output

60

15. Problem Statement

A service center assigns a priority number to each service request. A smaller number represents a higher priority. However, requests having the same priority must also be processed.

Using a PriorityQueue, process all requests and calculate the sum of the first K priorities that are processed.

Input Format

The first line contains N.

The second line contains N priority numbers.

The third line contains K.

Output Format

Print the sum of the first K priorities removed from the PriorityQueue.

Code Constraints

$1 \leq K \leq N \leq 1000$

$1 \leq \text{Priority} \leq 100000$

Sample Input

7

18 5 12 30 9 25 3

3

Sample Output

17

16. Problem Statement

A software application needs a reusable container capable of storing values of different data types. Create a generic class named `Box<T>` that stores a value and provides a method `getValue()` to retrieve it.

The program receives an integer and a decimal value. Create appropriate generic objects and display both values.

Input Format

The first line contains an integer.

The second line contains a decimal value.

Output Format

Print the integer on the first line and the decimal value rounded to two decimal places on the second line.

Sample Input

250

45.75

Sample Output

250

45.75

17. Problem Statement

A banking application accepts a withdrawal amount from a customer's account. A withdrawal is valid only when the amount is positive and does not exceed the available balance.

Write a Java program using try-catch-finally to perform the withdrawal. Handle invalid withdrawal amounts and insufficient balance without terminating the program unexpectedly.

If the withdrawal is valid, print the remaining balance.

If the amount is invalid, print Invalid Amount.

If the amount exceeds the balance, print Insufficient Balance.

Input Format

The first line contains the account balance.

The second line contains the withdrawal amount.

Output Format

Print the remaining balance or the appropriate error message.

Code Constraints

$0 \leq \text{Balance} \leq 1000000$

$-1000 \leq \text{Withdrawal Amount} \leq 1000000$

Sample Input

25000

7500

Sample Output

17500

18. Problem Statement

An online examination system accepts a student's score only when it is between 0 and 100. Create a custom exception named InvalidScoreException.

Create a method validateScore(int score) that throws the custom exception when the score is outside the valid range.

Write a Java program to validate the entered score and handle the exception.

Input Format

The input consists of a single integer representing the score.

Output Format

Print the score if it is valid.

Otherwise print:

Invalid Score

Code Constraints

$-100 \leq \text{Score} \leq 150$

Sample Input

85

Sample Output

85

19. Problem Statement

An examination portal stores marks obtained by students. The faculty wants to identify the marks that are above the class average.

Using the Stream API, calculate the average marks and then display only the marks strictly greater than the average, preserving their original order.

If no mark is greater than the average, print -1.

Input Format

The first line contains N.

The second line contains N space-separated marks.

Output Format

Print the qualifying marks separated by a single space.

Code Constraints

$1 \leq N \leq 100$

$0 \leq \text{Marks} \leq 100$

Sample Input

6

40 55 70 80 65 50

Sample Output

70 80 65

20. Problem Statement

A delivery company records the distances travelled by its vehicles. Some distances may be repeated because multiple vehicles travel the same distance.

Using the Stream API, remove duplicate distances, sort the remaining distances in descending order, and display the top three distinct distances. If fewer than three distinct distances exist, display all available distinct distances.

Input Format

The first line contains N.

The second line contains N space-separated distances.

Output Format

Print the top three distinct distances in descending order.

Code Constraints

$1 \leq N \leq 1000$

$1 \leq \text{Distance} \leq 100000$

Sample Input

8

120 450 300 450 700 120 550 700

Sample Output

700 550 450

21. Problem Statement

A company's payroll system stores the salaries of its employees. The management wants to calculate the total salary of employees whose salary is greater than a specified threshold.

Using the Stream API, filter the qualifying salaries and use a reduction operation to calculate their total.

Input Format

The first line contains N.

The second line contains N salary values.

The third line contains the salary threshold.

Output Format

Print the total salary of all employees whose salary is strictly greater than the threshold.

Code Constraints

$1 \leq N \leq 1000$

$10000 \leq \text{Salary} \leq 1000000$

$10000 \leq \text{Threshold} \leq 1000000$

Sample Input

6

35000 52000 48000 75000 42000 90000

50000

Sample Output

217000

22. Problem Statement

An online store maintains product prices. The store wants to apply a category-based transformation to its prices using the Stream API.

For every price:

- If the price is at least 1000, apply a 15% discount.
- Otherwise, apply a 5% discount.

Use Stream API mapping operations to generate the final prices and display them rounded to the nearest integer.

Input Format

The first line contains N.

The second line contains N integer prices.

Output Format

Print the updated prices separated by a single space.

Code Constraints

$1 \leq N \leq 100$

$100 \leq \text{Price} \leq 100000$

Sample Input

6

500 1200 800 2000 1500 900

Sample Output

475 1020 760 1700 1275 855

23. Problem Statement

A university maintains student marks and wants to prepare a summary using the Stream API. Each mark must be classified into one of three categories:

- High $\rightarrow$ marks ≥ 75
- Medium $\rightarrow$ marks between 50 and 74
- Low $\rightarrow$ marks below 50

Use `Collectors.groupingBy()` to group the marks according to these categories and display the number of students in each category.

Input Format

The first line contains N .

The second line contains N space-separated marks.

Output Format

Print:

High: count

Medium: count

Low: count

on separate lines.

Code Constraints

$1 \leq N \leq 1000$

$0 \leq \text{Marks} \leq 100$

Sample Input

8

82 45 67 91 38 75 54 49

Sample Output

High: 3

Medium: 2

Low: 3

24. Problem Statement

A data analytics system receives N integer values. The system needs to calculate the sum of the squares of only the even values.

Using the Stream API, filter the even numbers, transform each value into its square, and use a reduction operation to calculate the final sum.

Input Format

The first line contains N .

The second line contains N integers.

Output Format

Print the sum of squares of all even values.

Code Constraints

$1 \leq N \leq 100$

$-1000 \leq \text{Value} \leq 1000$

Sample Input

6

3 4 6 5 8 7

Sample Output

116

25. Problem Statement

A data processing system receives two integers and uses the first integer as an array index and the second integer as a divisor.

Write a Java program using exception handling to manage both an invalid array index and division by zero. Use appropriate exception classes from Java's exception hierarchy and handle each situation separately.

If the index is invalid, print:

Invalid Index

If the divisor is zero, print:

Division By Zero

Otherwise, print the value at the specified index divided by the divisor.

Input Format

The first line contains N.

The second line contains N integers.

The third line contains the index.

The fourth line contains the divisor.

Output Format

Print the calculated result or the appropriate exception message.

Code Constraints

$$1 \leq N \leq 100$$

$$-1000 \leq \text{Array Element} \leq 1000$$

$$-100 \leq \text{Index} \leq 100$$

$$-100 \leq \text{Divisor} \leq 100$$

Sample Input

5

20 40 60 80 100

2

10

Sample Output

6

26. Problem Statement

A library management system stores information about the books borrowed by each member. Currently, all borrowed book names are stored together in a single field, separated by commas.

For example:

201 Java,Python,DBMS

This creates multiple values in one field and therefore violates the requirements of First Normal Form (1NF).

To normalize the data, develop a Java program that converts each member's record into 1NF format, where each record contains exactly one member and one borrowed book.

The program should:

- Separate the comma-separated book list into individual book names.
- Generate a separate record for every book borrowed by a member.
- Preserve the original order of members.
- Preserve the original order of books for each member.

The resulting records should have the structure:

MemberID Book

Input Format

- The first line contains an integer N, representing the number of member records.
- Each of the next N lines contains:
 - o Member ID
 - o Comma-separated list of borrowed books.

Constraints

- $1 \leq N \leq 50$
- $1 \leq \text{number of books per member} \leq 5$
- Member ID is a positive integer.
- Book names contain only alphabets.
- There are no spaces around commas.

Output Format

Print each book as an individual record in the following format:

MemberID Book

Maintain the same order as provided in the input.

Sample Input

3

201 Java,Python,DBMS

202 HTML,CSS

203 JavaScript,React

Sample Output

201 Java

201 Python

201 DBMS

202 HTML

202 CSS

203 JavaScript

203 React

27. Problem Statement

A hospital management system maintains patient treatment information in a single table containing both patient details and treatment-specific information:

PatientID PatientName TreatmentID TreatmentName DoctorName

Since a patient may receive multiple treatments, the patient's ID and name are repeated for every treatment record. This causes unnecessary duplication of patient information.

To restructure the data according to the principles of Second Normal Form (2NF), the hospital wants to create a patient-level summary in which each patient appears only once.

Develop a Java program using arrays and loops to process the given treatment records and:

Identify each unique patient using the Patient ID.

Store the patient's name only once in the summary.

Count the total number of different treatment records associated with each patient.

Preserve the order in which patients first appear in the input.

Display the patient ID, patient name, and total treatment count.

Input Format

The first line contains an integer N, representing the number of treatment records.

Each of the next N lines contains:

Patient ID

Patient Name

Treatment ID

Treatment Name

Doctor Name

Constraints

$1 \leq N \leq 50$

Patient ID and Treatment ID are positive integers.

Patient and doctor names contain no spaces.

The same patient-treatment combination does not occur more than once.

Output Format

For each unique patient, print:

PatientID PatientName TreatmentCount

Maintain the order of the patient's first appearance in the input.

Sample Input

6

501 Rahul 301 Checkup DrKumar

502 Neha 302 XRay DrSharma

501 Rahul 303 BloodTest DrMehta

503 Aman 301 Checkup DrKumar

502 Neha 304 MRI DrSingh

501 Rahul 305 ECG DrPatel

Sample Output

501 Rahul 3

502 Neha 2

503 Aman 1

28. Problem Statement

A college administration system maintains student enrollment records containing student and department information in the following structure:

StudentID StudentName DepartmentID DepartmentName

During database analysis, it is observed that the DepartmentName is determined by the DepartmentID, rather than directly by the StudentID. Therefore, the dependencies can be represented as:

StudentID $\rightarrow$ StudentName, DepartmentID

DepartmentID $\rightarrow$ DepartmentName

This creates a transitive dependency, which is addressed by Third Normal Form (3NF).

For this programming task, the college wants to generate a simplified student summary in which duplicate student records are removed.

Develop a Java program using arrays and loops only to:

Identify whether a student has already been processed using the StudentID.

Store each unique student only once.

Associate the student with the corresponding department name.

Preserve the order in which unique students first appear in the input.

Ignore duplicate records for an already processed student.

Input Format

The first line contains an integer N, representing the number of enrollment records.

Each of the next N lines contains:

Student ID

Student Name

Department ID

Department Name

Constraints

$1 \leq N \leq 50$

Student IDs and Department IDs are positive integers.

Student names and department names contain no spaces.

The same student may occur more than once.

Repeated records for the same student contain the same department information.

Output Format

For every unique student, print:

StudentID StudentName DepartmentName

Maintain the order in which the students first appear in the input.

Sample Input

6

201 Arjun 10 ComputerScience

202 Priya 20 Commerce

203 Karan 10 ComputerScience

201 Arjun 10 ComputerScience

204 Sneha 30 Mathematics

202 Priya 20 Commerce

Sample Output

201 Arjun ComputerScience

202 Priya Commerce

203 Karan ComputerScience

204 Sneha Mathematics

29. Problem Statement

A university classroom management system stores course allocation information in the following structure:

CourseID FacultyID FacultyName RoomNo

A faculty member can teach multiple courses, and a room can be used for multiple courses.

The university has the following functional dependencies:

CourseID $\rightarrow$ FacultyID

FacultyID $\rightarrow$ FacultyName

RoomNo $\rightarrow$ FacultyID

The system wants to create a simplified faculty allocation summary by keeping only the first occurrence of each FacultyID.

Develop a pure Java program using arrays and loops only to:

Read the given allocation records.

Identify whether a faculty member has already been processed using FacultyID.

Store each unique faculty member only once.

Store the corresponding FacultyName and RoomNo.

Ignore duplicate records for the same faculty.

Preserve the order in which faculty members first appear.

Display the unique faculty allocation records.

Input Format

The first line contains an integer N, representing the number of records.

Each of the next N lines contains:

CourseID FacultyID FacultyName RoomNo

Constraints

$1 \leq N \leq 50$

Course ID, Faculty ID, and Room Number are positive integers.

Faculty names contain no spaces.

The same faculty may occur in multiple records.

Duplicate faculty records contain the same faculty name.

Output Format

For every unique faculty member, print:

FacultyID FacultyName RoomNo

Maintain the order of first appearance.

Sample Input

7

101 501 Sharma 201

102 502 Mehta 202

103 501 Sharma 201

104 503 Gupta 203

105 502 Mehta 202

106 504 Singh 204

107 501 Sharma 201

Sample Output

501 Sharma 201

502 Mehta 202

503 Gupta 203

504 Singh 204

30. Problem Statement

A hotel reservation system records the date on which a guest checks in and the number of nights included in the reservation. The system also records how many nights the guest has already stayed.

Develop a Java program using the Java Date-Time API to process the reservation details and determine the reservation status.

The program must:

Calculate the checkout date by adding the complete reservation period to the check-in date.

Determine the number of nights remaining by subtracting the nights already stayed from the total number of reserved nights.

Extract and display the year and month from the calculated checkout date.

Use the `LocalDate` class for all date calculations.

Input Format

The input consists of three lines:

The first line contains the check-in date in the format `yyyy-MM-dd`.

The second line contains the total number of nights reserved.

The third line contains the number of nights already stayed.

Output Format

Print the following information on separate lines in the specified order:

Checkout date

Remaining nights

Year of the checkout date

Month of the checkout date

Code Constraints

The check-in date is valid.

$1 \leq \text{totalNights} \leq 365$

$0 \leq \text{stayedNights} < \text{totalNights}$

Year is between 2020 and 2030.

Sample Input

2026-08-10

30

12

Sample Output

2026-09-09

18

2026

9

31. Problem Statement

A college examination system records the date and starting time of an examination. The examination lasts for a specified number of minutes.

Develop a Java program using the Java Date-Time API to determine the examination's ending date and time.

The program must:

Read the examination date using `LocalDate`.

Read the examination start time using `LocalTime`.

Combine the date and time using `LocalDateTime`.

Add the given examination duration in minutes.

Display the resulting ending date and time.

Display the year, month, and hour of the examination end time.

Use `LocalDate`, `LocalTime`, and `LocalDateTime` for the required operations.

Input Format

The input consists of three lines:

The first line contains the examination date in the format yyyy-MM-dd.

The second line contains the examination start time in the format HH:mm.

The third line contains the examination duration in minutes.

Constraints

The examination date is valid.

Time is provided in 24-hour format.

$30 \leq \text{duration} \leq 300$.

Year is between 2020 and 2030.

Output Format

Print the following on separate lines:

Examination end date

Examination end time

Year of the end date

Month of the end date

Hour of the end time

Sample Input

2026-09-14

22:30

120

Sample Output

2026-09-15

00:30

2026

9

0

32. Problem Statement

A smart electricity monitoring system receives the power consumption of several devices. The system needs to process the readings using a functional programming approach.

For every reading, calculate the adjusted consumption using:

Adjusted Consumption = Reading $\times$ 1.10

Write a Java program that stores the readings in a list and uses a functional approach to process each reading and display the adjusted consumption rounded to two decimal places.

Input Format

The first line contains the number of devices.

The second line contains the power consumption readings separated by spaces.

Output Format

Print the adjusted consumption of each device separated by a space, with each value displayed up to two decimal places.

Code Constraints

$1 \leq N \leq 100$

$0 \leq \text{Reading} \leq 10000$

Sample Input

```
4
100 200 300 400
```

Sample Output

```
110.00 220.00 330.00 440.00
```

33. Problem Statement

A delivery company assigns a priority score to packages based on their weight. A package receives a priority score equal to its weight multiplied by 2 and increased by 10.

Write a Java program using a Lambda Expression to calculate the priority score of every package.

Input Format

The first line contains the number of packages.

The second line contains the package weights separated by spaces.

Output Format

Print the priority score of each package separated by spaces.

Code Constraints

$1 \leq N \leq 100$

$1 \leq \text{Weight} \leq 1000$

Sample Input

3

10 25 40

Sample Output

30 60 90

34. Problem Statement

A quality-control system checks the temperature of machines. A machine is considered safe when its temperature is between 20 and 80 degrees Celsius, inclusive.

Create a Java program using a functional interface to determine whether each machine temperature is safe or unsafe.

Input Format

The first line contains the number of machines.

The second line contains the temperatures separated by spaces.

Output Format

Print Safe or Unsafe for each temperature, separated by spaces.

Code Constraints

$1 \leq N \leq 100$

$-100 \leq \text{Temperature} \leq 200$

Sample Input

5

25 80 90 15 50

Sample Output

Safe Safe Unsafe Unsafe Safe

35. Problem Statement

An employee management system searches for an employee name using an employee ID. If the employee exists, the system should display the employee name. If the ID is not found, it should display Employee Not Found.

Write a Java program using Optional to safely handle the possibility of a missing employee without directly checking for a null value.

Input Format

The first line contains the number of employees.

The next N lines contain employee ID and employee name.

The last line contains the ID to search.

Output Format

Print the employee name if the ID exists. Otherwise, print Employee Not Found.

Code Constraints

$1 \leq N \leq 100$

$1 \leq \text{Employee ID} \leq 100000$

Sample Input

```
3
101 Amit
102 Neha
103 Rahul
102
```

Sample Output

```
Neha
```

36. Problem Statement

A university examination system stores the examination date and needs to determine the number of days remaining until another specified date.

Write a Java program using the Java Date-Time API to calculate the number of days between two given dates.

Input Format

The first line contains the starting date in the format YYYY-MM-DD.

The second line contains the ending date in the format YYYY-MM-DD.

Output Format

Print the number of days between the two dates.

Code Constraints

Both dates are valid dates.

The ending date is not earlier than the starting date.

Sample Input

2026-09-01

2026-09-15

Sample Output

14

37. Problem Statement

A Java application creates several local integer variables inside a method. The program must demonstrate the difference between local variables and dynamically created objects.

Write a Java program containing a method that creates local primitive variables and an integer array using the new keyword. Print the values of the variables and array elements.

The objective is to demonstrate that local variables are associated with method execution while dynamically created objects are stored in heap memory.

Input Format

The first line contains three integers.

The second line contains three integers for the array.

Output Format

Print the three local values on the first line and the three array values on the second line.

Code Constraints

$-10000 \leq \text{Value} \leq 10000$

Sample Input

10 20 30

40 50 60

Sample Output

10 20 30

40 50 60

38. Problem Statement

A student-result application uses a method to calculate the total marks of a student. The method receives three marks and creates an integer array containing the marks.

Write a Java program that uses a separate method to calculate and print the total marks. The program should demonstrate method-local variables and an array object created during execution.

Input Format

The first line contains three subject marks.

Output Format

Print the total marks.

Code Constraints

$0 \leq \text{Marks} \leq 100$

Sample Input

75 82 90

Sample Output

247

39. Problem Statement

A document-processing application creates temporary String objects while processing documents. After the objects are no longer required, the application requests the JVM to perform garbage collection.

Write a Java program that creates a specified number of temporary objects, removes their references, requests garbage collection using the appropriate JVM method, and finally prints Garbage Collection Requested.

Input Format

The first line contains the number of temporary objects.

Output Format

Print:

Garbage Collection Requested

Code Constraints

$1 \leq N \leq 10000$

Sample Input

100

Sample Output

Garbage Collection Requested

40. Problem Statement

A payroll application calculates the salary of an employee using a simple arithmetic expression. Java source code is compiled into bytecode, which is executed by the JVM.

Write a Java program that reads the basic salary, allowance, and deduction and calculates the final salary.

Final Salary = Basic Salary + Allowance – Deduction

Input Format

The first line contains the basic salary.

The second line contains the allowance.

The third line contains the deduction.

Output Format

Print the final salary.

Code Constraints

$0 \leq \text{Basic Salary} \leq 1000000$

$0 \leq \text{Allowance} \leq 500000$

$0 \leq \text{Deduction} \leq 500000$

Sample Input

30000

5000

2000

Sample Output

33000

41. Problem Statement

A data-analysis application receives a large number of integers. To reduce unnecessary output operations, the application must first calculate all results and then print them together.

Write a Java program that reads N integers, calculates the square of every integer, stores the results in a StringBuilder, and prints all results at once.

Input Format

The first line contains N.

The second line contains N integers.

Output Format

Print the squares separated by spaces.

Code Constraints

$1 \leq N \leq 100000$

$-1000 \leq \text{Value} \leq 1000$

Sample Input

5

2 3 4 5 6

Sample Output

4 9 16 25 36

42. Problem Statement

A competitive-programming system receives a large number of integer values. The program must efficiently read all values and calculate their sum.

Write a Java program using buffered input to efficiently read N integers and calculate their sum.

Input Format

The first line contains N.

The second line contains N integers separated by spaces.

Output Format

Print the sum of all integers.

Code Constraints

$1 \leq N \leq 100000$

$0 \leq \text{Value} \leq 100000$

Sample Input

5

10 20 30 40 50

Sample Output

150

43. Problem Statement

A weather-monitoring system receives temperature readings continuously. Instead of printing each processed reading immediately, the system should construct the complete report using buffering.

For each temperature, calculate:

Processed Temperature = Temperature + 2

Use `BufferedReader` for input and `StringBuilder` for constructing the output.

Input Format

The first line contains N.

The second line contains N temperature readings.

Output Format

Print the processed temperatures separated by spaces.

Code Constraints

$1 \leq N \leq 100000$

$-100 \leq \text{Temperature} \leq 100$

Sample Input

4

20 25 30 35

Sample Output

22 27 32 37

44. Problem Statement

A billing application calculates the total cost of purchased items. Due to an error in the existing logic, the total is incorrectly calculated.

The correct formula is:

Total = Quantity × Price

Write a Java program with the corrected logic to calculate the total cost of each item and print the final total.

Input Format

The first line contains N.

Each of the next N lines contains quantity and price.

Output Format

Print the total cost of all items.

Code Constraints

$1 \leq N \leq 100$

$1 \leq \text{Quantity} \leq 1000$

$1 \leq \text{Price} \leq 100000$

Sample Input

3

2 100

5 50

3 200

Sample Output

1350

45. Problem Statement

A parking-management system calculates the parking fee based on the number of hours a vehicle stays in the parking area.

The fee rules are:

0 hours $\rightarrow$ 0

1 to 2 hours $\rightarrow$ 20 per hour

More than 2 hours $\rightarrow$ 40 per hour

Write a Java program that correctly handles normal cases as well as the edge cases of zero hours and exactly two hours.

Input Format

The first line contains the number of parking hours.

Output Format

Print the parking fee.

Code Constraints

$0 \leq \text{Hours} \leq 100$

Sample Input

2

Sample Output

40

46. Problem Statement

A financial application needs to apply different operations to an amount. The supported operations are addition of a fixed value and multiplication by a fixed value.

Write a Java program that passes a functional interface as an argument to a method. The method should apply the received operation to the given amount.

Input Format

The first line contains the amount.

The second line contains the operation value.

The third line contains the operation type: ADD or MULTIPLY.

Output Format

Print the resulting amount.

Code Constraints

$0 \leq \text{Amount} \leq 100000$

$0 \leq \text{Value} \leq 1000$

Sample Input

1000

50

ADD

Sample Output

1050

47. Problem Statement

A student-information system stores the names of students. The system needs to print each student name in uppercase.

Write a Java program using a method reference to refer to the appropriate String method for converting each name to uppercase.

Input Format

The first line contains N.

The second line contains N student names separated by spaces.

Output Format

Print all student names in uppercase separated by spaces.

Code Constraints

$1 \leq N \leq 100$

Each name contains only alphabetic characters.

Sample Input

3

amit neha rahul

Sample Output

AMIT NEHA RAHUL

48. Problem Statement

A warehouse system evaluates product quantities using different rules. One rule determines whether a product quantity is greater than 50, while another calculates the quantity after adding a safety stock of 10 units.

Write a Java program that uses functional interfaces and passes functions to separate methods to perform these operations.

Input Format

The first line contains N.

The second line contains N product quantities.

Output Format

First print the number of products having quantity greater than 50.

On the second line, print the updated quantities after adding 10.

Code Constraints

$1 \leq N \leq 100$

$0 \leq \text{Quantity} \leq 1000$

Sample Input

5

20 60 45 80 100

Sample Output

3

30 70 55 90 110

49. Problem Statement

A railway ticketing system processes a number of ticket requests using a separate thread. The system should start the ticket-processing thread, allow it to process all requests, and wait for the thread to complete before the main program continues.

Write a Java program that creates a thread to process the given number of ticket requests. The thread should print the number of processed requests. The main thread must wait for the ticket-processing thread to complete using `join()`.

Input Format

The first line contains the number of ticket requests.

Output Format

The thread should print the number of processed tickets. After the thread completes, print Processing Completed.

Code Constraints

$0 \leq N \leq 1000$

Sample Input

5

Sample Output

Tickets Processed: 5

Processing Completed

50. Problem Statement

A food-delivery application processes customer orders in a separate thread. Each order has an order ID, and the application must create a thread by extending the Thread class.

Write a Java program that creates a class extending Thread. The thread should process the given order and print its order ID.

Input Format

The first line contains the order ID.

Output Format

Print:

Order <ID> Processing

Code Constraints

$1 \leq \text{Order ID} \leq 100000$

Sample Input

105

Sample Output

Order 105 Processing

51. Problem Statement

A hospital monitoring system receives heart-rate readings from a patient-monitoring device. The readings must be processed by a separate thread created using the Runnable interface.

Write a Java program that implements Runnable and creates a thread to display each heart-rate reading.

Input Format

The first line contains the number of readings.

The second line contains N heart-rate readings separated by spaces.

Output Format

For every reading, print:

Monitoring: <reading>

Each reading should be printed on a separate line.

Code Constraints

$1 \leq N \leq 100$

$40 \leq \text{Reading} \leq 200$

Sample Input

3

72 80 76

Sample Output

Monitoring: 72

Monitoring: 80

Monitoring: 76

52. Problem Statement

A bank account is accessed by two withdrawal requests running in separate threads. Both requests operate on the same account balance.

Write a Java program using a synchronized withdrawal method so that only one thread can modify the account balance at a time.

If sufficient balance is available, the amount should be withdrawn. Otherwise, print Insufficient Balance.

Input Format

The first line contains the initial account balance.

The second line contains the withdrawal amount requested by the first thread.

The third line contains the withdrawal amount requested by the second thread.

Output Format

If a withdrawal cannot be completed, print Insufficient Balance.

After both threads finish, print:

Final Balance: <balance>

Code Constraints

$0 \leq \text{Balance} \leq 1000000$

$0 \leq \text{Withdrawal Amount} \leq 1000000$

Sample Input

10000

3000

4000

Sample Output

Final Balance: 3000

53. Problem Statement

A movie-ticket booking system maintains a shared number of available tickets. Two booking threads attempt to reserve tickets simultaneously.

Write a Java program that uses a synchronized block to protect the critical section where tickets are booked.

Each successful booking decreases the available ticket count by one. If there are not enough tickets, no additional tickets should be booked.

Input Format

The first line contains the initial number of tickets.

The second line contains the number of booking requests.

Output Format

Print:

Tickets Remaining: <number>

Code Constraints

$0 \leq \text{Tickets} \leq 10000$

$0 \leq \text{Requests} \leq 10000$

Sample Input

10

6

Sample Output

Tickets Remaining: 4

54. Problem Statement

A website maintains a shared visitor counter. Two threads represent two groups of visitors, and each thread increments the same counter.

Without proper synchronization, simultaneous updates can cause a race condition and produce an incorrect count.

Write a Java program that safely updates the shared counter using synchronization so that every visitor is counted.

Input Format

The first line contains the number of visitors handled by the first thread.

The second line contains the number of visitors handled by the second thread.

Output Format

Print:

Total Visitors: <number>

Code Constraints

$0 \leq N \leq 100000$

Sample Input

5000

7000

Sample Output

Total Visitors: 12000

55. Problem Statement

A manufacturing system has two shared resources required by two processing threads. If the threads acquire the resources in different orders, they may wait indefinitely for each other, resulting in a deadlock.

Write a Java program that uses two shared resources and ensures that both threads acquire the resources in the same order. This prevents deadlock and allows both operations to complete successfully.

Input Format

The first line contains the number of units processed by Thread 1.

The second line contains the number of units processed by Thread 2.

Output Format

Print:

Thread 1 Completed

Thread 2 Completed

Production Completed

Code Constraints

$1 \leq N \leq 1000$

Sample Input

10

20

Sample Output

Thread 1 Completed

Thread 2 Completed

Production Completed

56. Problem Statement

A communication system has two threads that need to send messages through a shared communication channel. The threads repeatedly give way to each other instead of proceeding, which can result in a livelock.

Write a Java program that simulates a limited number of retry attempts. After the specified number of attempts, the communication should proceed successfully.

Input Format

The first line contains the maximum number of retry attempts.

Output Format

Print:

Communication Started

For every unsuccessful retry, print:

Retrying

When communication succeeds, print:

Message Sent Successfully

Communication Completed

Code Constraints

$1 \leq \text{Attempts} \leq 10$

Sample Input

3

Sample Output

Communication Started

Retrying

Retrying

Message Sent Successfully

Communication Completed

57. Problem Statement

An online examination system maintains a shared counter representing the number of submitted answers. Multiple student threads may submit answers at the same time.

Write a Java program using a synchronized method to ensure that every submission is counted correctly.

Input Format

The first line contains the number of submissions from the first group of students.

The second line contains the number of submissions from the second group.

Output Format

Print:

Total Submissions: <number>

Code Constraints

$0 \leq N \leq 100000$

Sample Input

2500

3500

Sample Output

Total Submissions: 6000

58. Problem Statement

A server receives login requests from multiple threads. Each thread generates a set of login IDs, and the IDs must be stored in a thread-safe collection because multiple threads may add data concurrently.

Write a Java program using a concurrent collection to store the login IDs generated by two threads. After both threads finish, print the total number of stored login records.

Input Format

The first line contains the number of login records generated by the first thread.

The second line contains the number of login records generated by the second thread.

Output Format

Print:

Total Login Records: <number>

Code Constraints

$0 \leq N \leq 100000$

Sample Input

4000

6000

Sample Output

Total Login Records: 10000

59. Problem Statement

A university is developing a student management system. The database must store student information while ensuring that important data follows appropriate rules.

Create a MySQL table named Students with the following fields:

- student_id
- student_name
- email
- age
- course

The table must ensure that student_id is the primary key, student_name cannot be NULL, email is unique, and age must be between 17 and 60.

After creating the table, insert the given student records and display all records in ascending order of student_id.

Input Format

The first line contains an integer N, representing the number of students.

The next N lines contain:

student_id student_name email age course

Output Format

Display all student records in ascending order of student_id.

Constraints

- $1 \leq N \leq 100$
- $17 \leq \text{age} \leq 60$
- student_id is unique.
- Email addresses are unique.
- Student names and course names contain no spaces.

Sample Input

3

101 Amit amit@gmail.com 20 CSE

102 Neha neha@gmail.com 21 ECE

103 Rahul rahul@gmail.com 22 CSE

Sample Output

101 Amit amit@gmail.com 20 CSE
102 Neha neha@gmail.com 21 ECE
103 Rahul rahul@gmail.com 22 CSE

60. Problem Statement

A company maintains employee records containing employee ID, employee name, department, and salary. The management wants to provide a 10% salary increment to all employees working in the CSE department.

Update the salary of every CSE employee by 10% and display the employee name, department, and updated salary.

Input Format

The first line contains an integer N, representing the number of employees.

The next N lines contain:

employee_id employee_name department salary

The final line contains the department for which the salary increment must be applied.

Output Format

Display:

employee_name department updated_salary

for all employees in ascending order of employee_id.

Constraints

- $1 \leq N \leq 100$
- Salary is a positive integer.
- Employee names and department names contain no spaces.
- The increment is exactly 10%.

Sample Input

4
101 Amit CSE 50000
102 Neha ECE 45000
103 Rahul CSE 60000

104 Simran IT 55000

CSE

Sample Output

Amit CSE 55000.00

Neha ECE 45000.00

Rahul CSE 66000.00

Simran IT 55000.00

61. Problem Statement

A library maintains details of its books, including book ID, title, author, and price. The librarian wants to remove all books whose price is below a specified minimum price.

Delete all books whose price is less than the given minimum price and display the remaining books in ascending order of book_id.

Input Format

The first line contains an integer N, representing the number of books.

The next N lines contain:

book_id title author price

The final line contains the minimum acceptable price.

Output Format

Display the remaining books as:

book_id title author price

in ascending order of book_id.

Constraints

- $1 \leq N \leq 100$
- Price is a positive integer.
- Book titles and author names contain no spaces.

Sample Input

5

101 Java James 550

102 SQL Korth 250

103 Python Rossum 650

104 Networks Tanenbaum 300

105 OS Galvin 200

300

Sample Output

101 Java James 550

103 Python Rossum 650

104 Networks Tanenbaum 300

62. Problem Statement

A university stores student information along with the department in which each student is enrolled. The administration wants to know how many students belong to each department.

Write a MySQL query that groups students according to their department and calculates the number of students in each department.

Input Format

The first line contains an integer N.

The next N lines contain:

student_id student_name department

Output Format

Display:

department student_count

in alphabetical order of department.

Constraints

- $1 \leq N \leq 100$
- Each student belongs to one department.
- Department names contain no spaces.

Sample Input

6

101 Amit CSE

102 Neha ECE

103 Rahul CSE

104 Simran IT

105 Karan CSE

106 Priya ECE

Sample Output

CSE 3

ECE 2

IT 1

63. Problem Statement

An online shopping system stores orders placed by customers. The company wants to calculate the total amount spent by each customer.

Write a MySQL query that groups the orders by customer and calculates the total order amount for every customer.

Input Format

The first line contains an integer N.

The next N lines contain:

order_id customer_id amount

Output Format

Display:

customer_id total_amount

in ascending order of customer_id.

Constraints

- $1 \leq N \leq 100$
- Amount is a positive integer.

Sample Input

5

1 101 1200

2 102 800

3 101 1500

4 103 2000

5 102 700

Sample Output

101 2700

102 1500

103 2000

64. Problem Statement

A university maintains examination marks for different subjects. The examination department wants to calculate the average marks obtained in each subject.

Write a MySQL query that groups the records by subject and calculates the average marks.

Input Format

The first line contains an integer N.

The next N lines contain:

student_id subject marks

Output Format

Display:

subject average_marks

with the average rounded to two decimal places. Display subjects in alphabetical order.

Constraints

- $1 \leq N \leq 100$
- $0 \leq \text{marks} \leq 100$
- Subject names contain no spaces.

Sample Input

6

101 Java 80

102 Java 90

103 DBMS 70

104 DBMS 80

105 Java 100

106 DBMS 90

Sample Output

DBMS 80.00

Java 90.00

65. Problem Statement

An online shopping company wants to identify customers who have placed at least three orders.

Write a MySQL query that groups orders by customer and displays only those customers whose total number of orders is three or more.

Input Format

The first line contains an integer N.

The next N lines contain:

order_id customer_id amount

Output Format

Display:

customer_id order_count

for customers having at least three orders.

Constraints

- $1 \leq N \leq 200$
- Order IDs are unique.
- Customer IDs are positive integers.

Sample Input

7

1 101 500

2 102 700

3 101 400

4 103 900

5 101 600

6 102 300

7 101 800

Sample Output

101 4

66. Problem Statement

A company wants to identify employees whose salary is greater than the average salary of all employees.

Write a MySQL query using a nested query to find these employees.

Input Format

The first line contains an integer N.

The next N lines contain:

employee_id employee_name salary

Output Format

Display:

employee_name salary

for employees whose salary is greater than the overall average salary. Display them in descending order of salary.

Constraints

- $1 \leq N \leq 100$
- Salary is a positive integer.
- Employee names contain no spaces.

Sample Input

5

101 Amit 40000

102 Neha 60000

103 Rahul 50000

104 Simran 80000

105 Karan 30000

Sample Output

Simran 80000

Neha 60000

67. Problem Statement

A university stores student details in one table and department details in another table.

The Students table contains student information and Departments contains department information. Write a MySQL query using an inner join to display each student's name along with the corresponding department name.

Input Format

The database contains the following information:

Students

student_id

student_name

department_id

Departments

department_id

department_name

Output Format

Display:

student_name department_name

in ascending order of student_id.

Constraints

- Every student has a valid department.
- department_id uniquely identifies a department.

Sample Input

Students

101 Amit 10

102 Neha 20

103 Rahul 10

Departments

10 CSE

20 ECE

Sample Output

Amit CSE

Neha ECE

Rahul CSE

68. Problem Statement

A company wants to generate a report containing every employee, including employees who have not yet been assigned to a department.

Write a MySQL query using a left join to display the employee name and department name. If an employee has no department, the department value should appear as NULL.

Input Format

The database contains:

Employees

employee_id

employee_name

department_id

Departments

department_id

department_name

Output Format

Display:

employee_name department_name

in ascending order of employee_id.

Sample Input

Employees

101 Amit 10

102 Neha NULL

103 Rahul 20

Departments

10 CSE

20 ECE

Sample Output

Amit CSE

Neha NULL

Rahul ECE

69. Problem Statement

A company stores managers and employees in the same Employees table. Each employee record contains the ID of their manager.

Write a MySQL query using a self join to display every employee along with the name of their manager.

Employees who do not have a manager should also be included.

Input Format

The database contains:

employee_id

employee_name

manager_id

Output Format

Display:

employee_name manager_name

in ascending order of employee ID.

Sample Input

101 Amit NULL

102 Neha 101

103 Rahul 101

104 Simran 102

Sample Output

Amit NULL

Neha Amit

Rahul Amit

Simran Neha

70. Problem Statement

An online store wants to find the product or products having the highest price.

Write a MySQL query using a subquery to display all products whose price is equal to the maximum product price.

Input Format

The first line contains an integer N.

The next N lines contain:

product_id product_name price

Output Format

Display:

product_name price

for every product having the maximum price.

Constraints

- $1 \leq N \leq 100$
- Price is a positive integer.
- More than one product may have the same maximum price.

Sample Input

5

101 Laptop 60000

102 Phone 40000

103 Monitor 15000

104 Tablet 30000

105 Camera 60000

Sample Output

Laptop 60000

Camera 60000